ECommerce Api

Introduction:

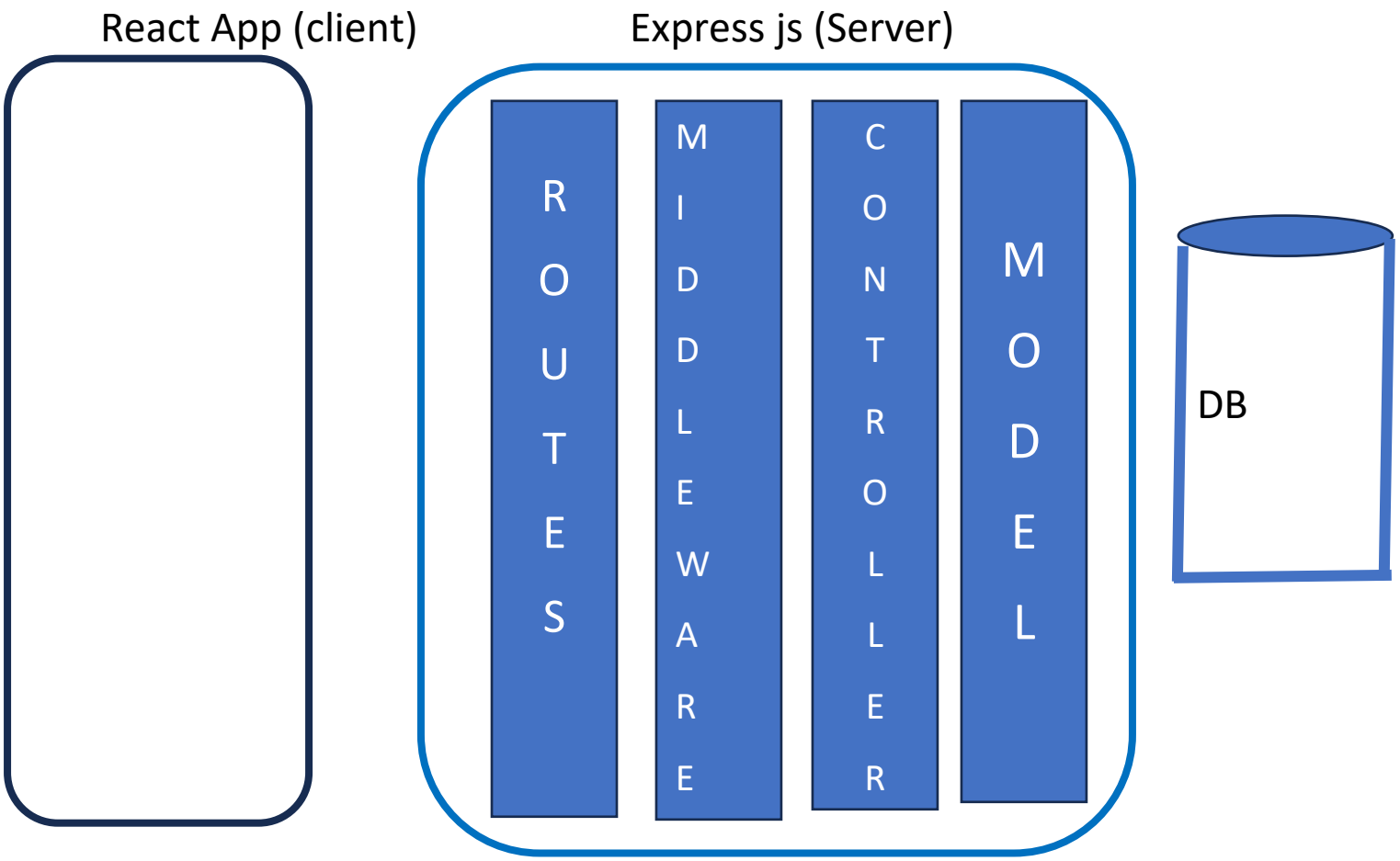
Build a scalable and user-friendly ecommerce platform that allows customers to browse, purchase, and track products online, while enabling sellers to manage inventory and sales efficiently.

Software Installation:

Please make sure you have installed below softwares:

1. Visual Studio Code
2. Node 24.x
3. Jest , supertest
4. MySQL 8.x with workbench
5. Postman

Architecture diagram:



REST API specifications:

Sr. No.	URL + METHOD	Functionality	Request Cookie	Request body	Response body	Response Staus	Query string
1	/users/authenticate POST	Logins a user		{"username": "a", "password": "a"}	{"username": "a", “roles”:[‘USER’,’ADMIN’]}	200	
2	/users/authenticate POST	Logins a user		Wrong cerdentails {"username": "a", "password": "a"}	{message:”Bad Credentials”}	404	
3	/users/logout POST	Logs out a user	jwt-token		{message:”Log gout Successfully”}	200	
4	/users/logout POST	Logs out a user	jwt-token		{message:”Unauthorized”}	401,403	
5	/users/register POST	Registers an user		{"username": "a", "password": "a"}	{"username": "a", "roles": [‘USER’,’ADMIN’]}	201	

6	/users GET	Returns all users information	ADMIN jwt-token		[{"username": "a", "roles": [‘USER’,’ADMIN’]}]	200	
7	/users/profile/{username} GET	Returns a user information	jwt-token		{"username": "a", "roles": [‘USER’,’ADMIN’]}	200	
8	/products GET	Returns all products			[{“id:1,”price”: 100,”name”:”tv”}]	200	
9	/products/{id} GET	Returns a product			{“id:1,”price”: 100,”name”:”tv”}	200	
10	/products/{id} DELETE	Delete a product	ADMIN jwt-token			204	
11	/products POST	Create a new Product	ADMIN jwt-token	{”price”:100,”name”:”tv”}		201	

12	/orders GET	Retrieve all orders	ADMIN jwt-token		[{"id": 1, "username": "a", "productId": 1, "quantity": 1}]	200	
13	/orders/{username} GET	Retrieve all orders for a user	USER jwt-token		[{"id": 1, "username": "a", "productId": 1, "quantity": 1}]	200	
14	/orders POST	Place new order	USER jwt-token	{"username": "a", "productId": 1, "quantity": 1}	{"id": 1, "username": "a", "productId": 1, "quantity": 1}	201	
15	/orders/{id} DELETE	Cancel a specific order	auth-token			204	

Testing:

Write Unit and integration Test cases using jest , supertest for each endpoint

Database design:

Please find below the tables along with their CREATE script.

Note that developer is free to give different database names, table names, column names and add new columns.

Sr. No.	Database Name	Table/s	Create Table Script
1.	Ecommerceapp	tbl_user	CREATE TABLE `ecommerceapp_db`.`tbl_user` (`username` VARCHAR(45) NOT NULL,

			`password` VARCHAR(45) NOT NULL, PRIMARY KEY (`username`));
2	Ecommerceapp	tbl_role	CREATE TABLE `ecommerceapp_db`.`tbl_role` (`role` VARCHAR(45) NOT NULL, `username_fk` VARCHAR(45) NULL, INDEX `u_fk_idx` (`username_fk` ASC) VISIBLE, CONSTRAINT `u_fk` FOREIGN KEY (`username_fk`) REFERENCES `ecommerceapp_db`.`tbl_user` (`username`) ON DELETE NO ACTION ON UPDATE NO ACTION);
3.	Ecommerceapp	tbl_product	CREATE TABLE `ecommerceapp_db`.`tbl_product` (`id` INT NOT NULL AUTO_INCREMENT, `name` VARCHAR(45) NOT NULL, `price` DECIMAL NOT NULL, PRIMARY KEY (`id`));
4.	Ecommerceapp	tbl_order	CREATE TABLE tbl_order (order_id INT PRIMARY KEY AUTO_INCREMENT, username_fk VARCHAR(45) NOT NULL, product_id_fk INT NOT NULL, quantity INT NOT NULL, FOREIGN KEY (username_fk) REFERENCES tbl_user(username), FOREIGN KEY (product_id_fk) REFERENCES tbl_product(id));

Code submission

Participants need to push the final code into GITHUB repository using personal GIT account & share the GIT URL with us for code review.